Introduction to R and RStudio

Anders Jensen

11/11/2025

University of Liverpool

Introduction

For people working across various fields in University, one of the most common struggles we all face is working with and analysing data. There are many different types of software available for people to use, such as SPSS, Microsoft Excel, Origin, MATLAB and more. These tools will often provide users with a friendly interface designed with the sole purpose of simplifying tasks for users. This is great for beginners; however, the simplicity also leads to limitations.

R programming is a coding language specifically designed for data analysis and statistical computing. Along with RStudio, which is a user interface to make R function more effectively, R is capable of performing every task in the aforementioned software and much more. You may think that learning an entire coding language is a bit overkill; however, the advantages you will gain in terms of both employability and efficiency cannot be understated. The aim of this document is to give you a brief introduction to R programming so that the basics are covered. One document is not enough to go through every aspect of RStudio; however, this will lay the foundation for further work in R.

Key Advantages of Using R Programming

- **Versatility**: Easy to work with various types of data from simple descriptive statistics to complex machine learning algorithms.
- **Efficiency**: Repetitive tasks can be streamlined to increase your productivity.
- **Collaboration**: You can implement code from other people into your own work and then share your code with others for reproducible analysis pipelines.
- **Customisation**: R can allow users to customise aspects of the analysis and visualisation, which may be more difficult with other software.

Installing R and RStudio

The first step into using R is making sure you can access both R and RStudio. R is an opensource programming language so everything is freely available to download online or if you are working on campus you can use the Install University Applications app on the desktop. RStudio is also required and is an interface that makes running R code so much easier. Strictly speaking it isn't 100% needed but you would be crazy not to use it.

On Campus

1) Click on he install university applications icon
2) Search for R and you should see both R and RStudio
3) Install the latest version of R
4) Once R is installed then install the latest version of RStudio
5) The you should be able to search for the RStudio icon (Blue with a white R)

Off Campus

1) **Click here** to begin installing R
2) Make sure to choose the option for your computer
3) Once R has installed then **click here** to install RStudio
4) Follow the download instructions until both R and RStudio are installed

Getting to Know RStudio

When you first open RStudio you should see the page split into three compartments. The console (Left-side), the environment (Top-right) and finally the files, plots, help tab (Bottom-right).

Each of these sections of rstudio are important for different reasons.

Console: This is where your code is run, you can write your code directly in here, however as you will see later it is easier to make a new script so that your work can be saved for a later date. This is also where any error messages will be shown.

Environment: As you can see this region has multiple windows however the environment is the most important. This is where you can store different objects (data frames, variables, functions, etc.).

Files, plots, help: Similarly to the environment there are other tabs in this window, however the files, plots and help windows are the most important.

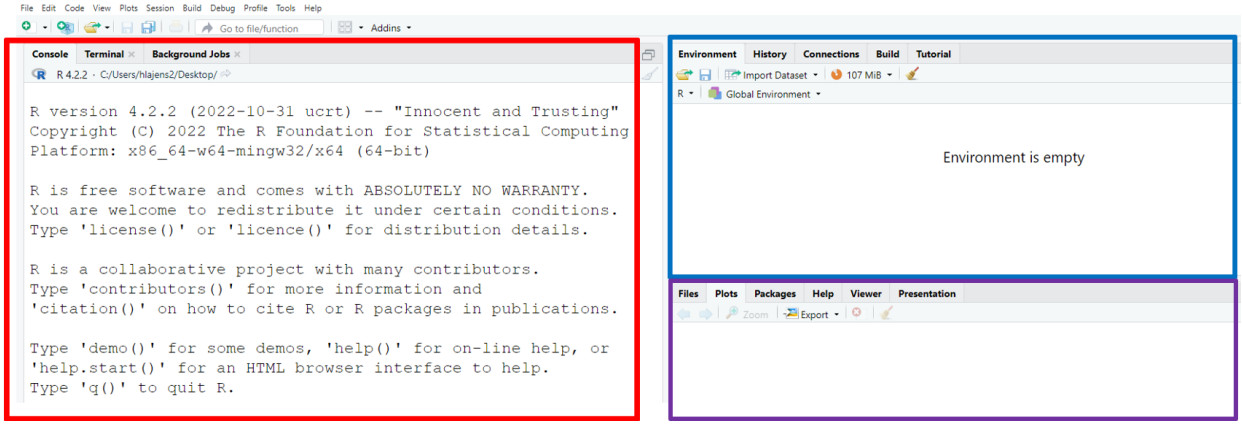


Figure 1: The rstudio interface

Starting your first R script

To do this, simply click on the small page icon on the top left of your screen (circled below). Then click "R script". The keyboard shortcut for this is Ctrl+Shift+N.

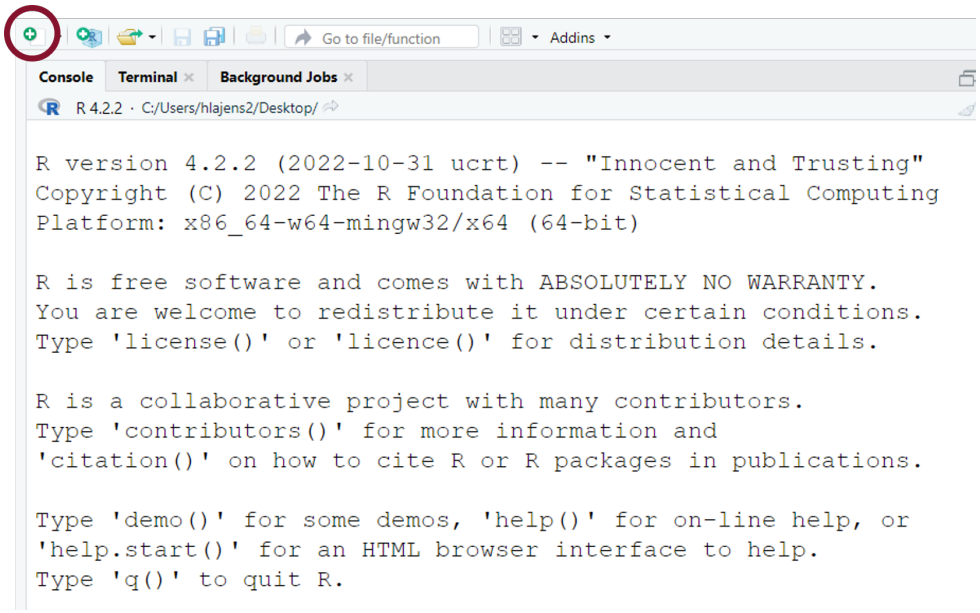


Figure 2: Starting an R script

As soon as you click this you should see a fourth window appear with the name **Untitled1**. This is your R Script. The benefits of working with R scripts is that it is easy to save for future use.

Make sure when saving you save the .R file as something that explains the work well so your life is easier in the future.

Writing Your First R Code

Now that you have an empty R script you can begin writing in some code. A useful tip before you start is to remember the importance of annotating your code. By using the # symbol you can write notes as you are going along, which makes your life much easier when you come back to your code in the future.

After you have done this try and do a simple addition calculation, example below.

```
# Anders Jensen
10+10
```

Now that you have written your code you need to run it. There are two ways to do this, but first make sure you click on the line of code you want to run. Then press Ctrl+Enter, or if you want to click the small button that says run you can do that, but it's not nearly as cool.

As a challenge, try and work out the following calculations using R code.

1) 8 x 19
2) 90 divided by 10
3) 9 to the power of 3
4) Square root of 100
5) Logarithm of 100

Hint: *, /, ^, sqrt(), log()

Answers are on the next page

```
8*19
```

```
## [1] 152
```

```
90/10
```

```
## [1] 9
```

```
9^3
```

```
## [1] 729
```

```
sqrt(100)
```

```
## [1] 10
```

```
log(100)
```

```
## [1] 4.60517
```

Saving Information to the Environment

Now that you can perform simple calculations using RStudio we now need to learn how to save the results of into our environment so that we can refer to them later.

To do this we need to use this arrow **<-**. You can either type this out manually or you can press **Alt+-**. Before performing your calculation, assign the output a name, then write the arrow, then your R code (example below).

```
answer <- sqrt(18*9/log(10^2))
```

Rules of Naming

When assigning a name to an object it actually doesn't matter what you call it, I chose answer above because it makes sense, but it would have worked fine if I had written the word dog. Generally you want to try and name things in a way that makes sense, however there are some rules.

1) Try to avoid numbers, especially at the start or beginning.
2) Don't use spaces, if you need to have a space use an underscore.
3) Avoid special characters (?, !, :) as they have other roles and can interfere with your code.

Interacting with the environment

Using the calculations examples from before, try to save all the answers into your environment with a given name.

Once you have done this, try to use R code to work out the sum of all the previous answers (example below)

```r
multiplication <- 8*19

division <- 90/10

cube <- 9^3

square_root <- sqrt(100)

logs <- log(100)

multiplication+division+cube+square_root+logs
```

```
## [1] 904.6052
```

So from this you can see once something is named and saved into the environment you can later refer to this object in other lines of coding simply by writing the name you assign.

Reading Data Into RStudio

Hopefully at this stage you are more comfortable with the rstudio interface and how to run lines of code and how to then save information into your working environment. The next and one of the most important steps is to be able to read in different types of data into rstudio.

For this workbook you will learn how to read in the two most common types of files, which are used to store data.

CSV – Comma separated values

XLSX – Excel spreadsheet (XML)

When you save a file containing data you should make sure about the type of file it is saved as. This is because the process of reading it into rstudio can vary.

Setting the Working Directory

In order to make sure that rstudio can find your saved file you first need to make sure it is looking in the correct place. The best way to think of a directory is like an address:

Continent -> Country -> City -> Street name -> House number

C drive -> Users -> hlajens2 -> Desktop -> Introduction to R

It is basically saying to rstudio that you need to look in this folder to find the correct document. In the same was as you wouldn't tell a delivery driver to deliver something to America, you would need to be more specific. To ensure there are no problems, make a new folder called "Introduction to R" on your desktop and save a csv file and an xlsx file that you want to try this with.

The easiest way to set your working directory is to click Session then click Set Working Directory then click Choose directory as seen in the image below (Or Ctrl+Shift+H).

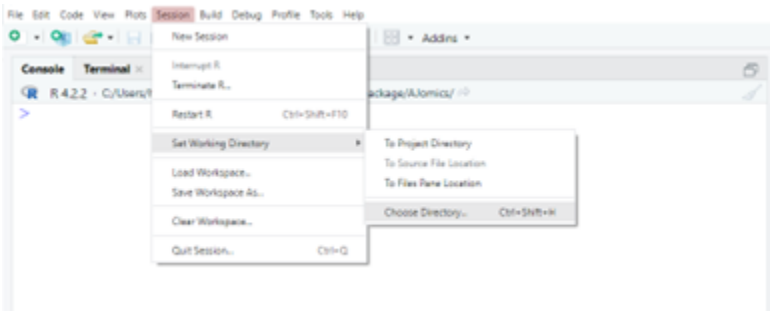


Figure 3: setting the directory

In this workbook I will be using the mtcars dataset. To download this dataset simply run the following code after setting your working directory. Once you run this code the dataset will save in the same place as your directory. If you then open this file with microsoft excel you can also save it as an xlsx file to practice with.

```
data("mtcars")

write.csv(x = mtcars, file = "mtcars.csv")
```

Attribute	Description	Type
mpg	Miles Per Gallon	Continuous
cyl	Number of cylinders	Nominal
disp	Displacement	Continuous
hp	Horse Power	Continuous
drat	Real Wheel Axle Ratio	Continuous
Wt	Weight	Continuous
qsec	Time for 0.25 mile	Continuous
vs	V/S	Nominal
am	Transmission type	Nominal
gear	Number of forward gears	Ordinal
carb	Number of carburetors	Ordinal

Figure 4: Summary of mtcars dataset

Reading a CSV File

Csv files are commonly used when transferring data and are often the required format for many different kinds of software. Large datasets can be stored compactly, allowing a more efficient transfer of data.

To read a csv file into rstudio we are going to use the read.csv function, which does exactly what it says on the tin

The following code will save the mtcars dataset as a data frame called df into your environment.

```
df <- read.csv(file = "mtcars.csv")

head(df, n = 3)
```

```
##               X  mpg cyl disp  hp drat    wt  qsec vs am gear carb
## 1     Mazda RX4 21.0   6  160 110 3.90 2.620 16.46  0  1    4    4
## 2 Mazda RX4 Wag 21.0   6  160 110 3.90 2.875 17.02  0  1    4    4
## 3    Datsun 710 22.8   4  108  93 3.85 2.320 18.61  1  1    4    1
```

I then used the head() function to observe the data but you can also do this by clicking in the environment where it says df.

Reading a XLSX File

To read data into rstudio in the xlsx format you first need to install a package that is capable of opening these types of files. Packages are just a collection of custom functions that can be used for a variety of tasks. There are many available packages for reading in .xlsx files but the one we will be using is called "readxl"

To install a package we need to run the following code:

```
install.packages("readxl")
library(readxl)
```

As you can see you need to install the package and then load it into rstudio using the library function. You do not need to install packages every single time, however if you close down rstudio and re-open it you will need to load the package again.

XLSX files are slower than csv files, however are useful when your data is stored across mulitple sheets of the same file.

To then read the file into rstudio you need to use read_xlsx function. As you can see below I have specified the file name (path) and the sheet number (sheet).

```
df <- read_xlsx(path = "mtcars.xlsx", sheet = 1)
```

Working With Data

Before going on any further there are some very useful functions that can be used to summarise a data frame in rstudio. The **str** function allows you to look at the types of data that exists in your columns (Numeric, Factor, Character, Integer, Logical, etc.) The **head** function allows you to see a certain number of rows of the dataframe to see how they look and also to show the column names.

Try the following code and observe the outputs in the console.

```
# data structure
str(df)

# observe 10 rows
head(df, n = 10)
```

Now that you have a dataset in rstudio and are happy with the columns we can begin a variety of different tasks. There are too many things to all be covered in this document so we will work through five specific common examples.

Subset Based on Condition

To do this we need to use the **subset** function.

Let's say we only want rows where the mpg (miles per gallon) is above 15.0 mpg.

```
# do the subset
df_subset <- subset(df, mpg > 15)

# how many rows originally
nrow(df)
```

```
## [1] 32
```

```
# how many rows in the subset
nrow(df_subset)
```

```
## [1] 26
```

As you can see the > selects everything larger than 15.0 mpg You can use == if you want to select for an exact number If you have multiple conditions you can use the two symbols:

& - stands for 'And' | - stands for 'Or'

```
# mpg more than 15 and weight is less than 3
df_subset <- subset(df, mpg > 15 & wt < 3)

# mpg more than 15 or the car has 4 cylinders
df_subset <- subset(df, mpg > 15 | cyl == 4)
```

Selecting Columns

To do this we will use the select function from the dplyr package. Dplyr is an extremely useful package and is definitely worth spending some more time to learn. Let's say we want a new dataframe that only contains mpg and qsec (quarter mile time)

```
# install package
install.packages("dplyr")

# load package
library(dplyr)

# make new dataframe
df_selected <- dplyr::select(df, mpg, qsec)
```

Creating a New Column

To do this we will be using the **$** operator, which is very important when working with data.

Let's say we want to make a column that is the horsepower per 1000 lbs (weight, wt). to do this we need to do horsepower/wt

```
# make new column (method 1)
df$hpwt <- df$hp/df$wt

# make new (method 2 using dplyr)
df_new_column <- dplyr::mutate(df, hpwt = hp/wt)
```

Changing Column Types

In our mtcars example the am column is in binary where Transmission (0 = automatic, 1 = manual). This is fine but another way to show it is by using TRUE/FALSE statement. Let's say we want to make the am column a TRUE if its manual and FALSE if its automatic.

```
# True = manual (1), False = automatic (0)
df$am <- as.logical(df$am)

# check structure
str(df)
```

If you wanted the TRUE/FALSE statement to be the other way around so that automatic was true, simply use the ! operator before the mtcars inside the brackets. as.logical, as.character, as.numeric can also all be used.

Producing Vectors

If we want to save the information from a specific column and have that as a list of values rather than a dataset, then we can do this with the **$** operator in a similar way to previously.

Let's say we want to save all the names of the cars in the dataset.

```r
# extract information
carnames <- df$X

# observe information
cat(carnames)
```

```
## Mazda RX4 Mazda RX4 Wag Datsun 710 Hornet 4 Drive Hornet Sportabout Valiant Duster 360 Merc 240D Merc
```

These examples only provide a snippet of the tasks you may need to ever complete. The best way to solve these problems is knowing where to look. Datanovia, stackoverflow, Statology are all useful websites with helpful tutorials and quick fixes.

Statistics in RStudio

RStudio is designed for statistical analysis and this is where it really thrives against other coding languages.

We will start by working out simple descriptive statistics before moving on to statistical testing.

The following functions are extremely useful for this:

mean – Calculate mean

median – Calculate median

mode – Calculate mode

range – Calculates the range

sd – Calculates the standard deviation

sum – Calculates the sum

max – Calculates the highest value

min – Calculates the lowest value

Try to use this and what you have learnt so far to calculate descriptive statistics on the miles per gallon (mpg) of cars in the mtcars dataset.

Do automatic cars have a higher average mpg than manual cars? (Hint: Subset)

Also have a go at using other functions that may speed up the process:

summary(vector)

describe(vector) – Part of the Hmisc package

skim(vector) – Part of the skimR package

descry(vector) – Part of the summarytools package

Remember you will need to install and load the package before running the function.

Statistical Testing in RStudio

Statistical testing is the idea of using mathematical techniques to make inferences about the characteristics of a population with your sample data. We are assessing the likelihood of an observed difference being down to chance. We then use statistical testing to generate a p-value. The p-value can be used a threshold for when we would consider something significant. A commonly used threshold of significance is 0.05 meaning there is a 5% chance the difference is due to chance.

There are too many statistical tests to cover in this document so we will go through three of the most common:

1) Shapiro-Wilks normality testing
2) Two Sample t-test
3) Pearson's correlation test

For this task we are attempting to answer two main questions:

1) Do straight engines have a higher miles per gallon than v-shaped engines?
2) Is there a correlation between weight and miles per gallon?

Worked Example 1

Do straight engines have a higher miles per gallon than v-shaped engines ?

We are comparing one continous scale (miles per gallon) against two different groups (engine shape). This immediately tells us that we need to do a t-test of some kind.

To understand which t-test we are doing we need to look at the assumptions of a t-test.

Are the data independent? Yes they are different cars

Is the data normally distributed? Not sure so let's find out....

Normality testing

The shapiro-wilk test is a quantitative test to determine if data follows a normal distribution or not (i.e follows a bell-shaped curve on a histogram).

Using our threshold of 0.05 we would say that a p-value larger than 0.05 is normally distributed and anything lower is non-normally distributed.

To do this on rstudio we first need to subset mtcars into v-shaped and straight engines.

```
# subset our data
v_shaped <- subset(df, vs == 0)
straight <- subset(df, vs == 1)

# run shapiro test
shapiro.test(v_shaped$mpg)
shapiro.test(straight$mpg)
```

If you look in the console you will see the output and notice that for both tests the p-value is greater than 0.05 so we can say the data is normally distributed. It may seem the other way around to other statistical tests, however the null hypothesis is that the data is normally distributed.

Now that we know the data is normally distributed we can proceed with a parametric t-test. To do this we need to use the **t.test** function.

Performing a t-test

```
# perform t test
t.test(data = df, mpg~vs)
```

Once again if you look in your console you will have a result from the t-test. You should be able to see if the difference is significant or not. Remember $p<0.05$ = significant result

Because the p-value is smaller than 0.05 we can say there is a significant difference between the miles per gallon of straight engines and v-shaped engines. This may be down to other variables but from the data we have tested this conclusion is fair.

Worked Example 2

Is there a correlation between weight and miles per gallon?

For this question we want to look for an association between two continuous variables (weight and miles per gallon). Therefore we need to do some type of correlation analysis.

First of all we need to check the distribution to see if we need a Pearson (parametric) or Spearman's rank (non-parametric)

```
# run shapiro test
# mpg
shapiro.test(df$mpg)

# weight
shapiro.test(df$wt)
```

Depending on these results we then need to perform a correlation analysis. Remember that if $p < 0.05$ the data is not normally distributed and if $p > 0.05$ the data is normally distributed. If one out of two variables are not normally distributed then a non-parametric statistical test is recommended.

The following code shows you how to do both of the correlation tests.

```
# parametric (normally distributed)
cor.test(df$mpg, df$wt, method = "pearson")

# non-parametric (not normally distributed)
cor.test(df$mpg, df$wt, method = "spearman")
```

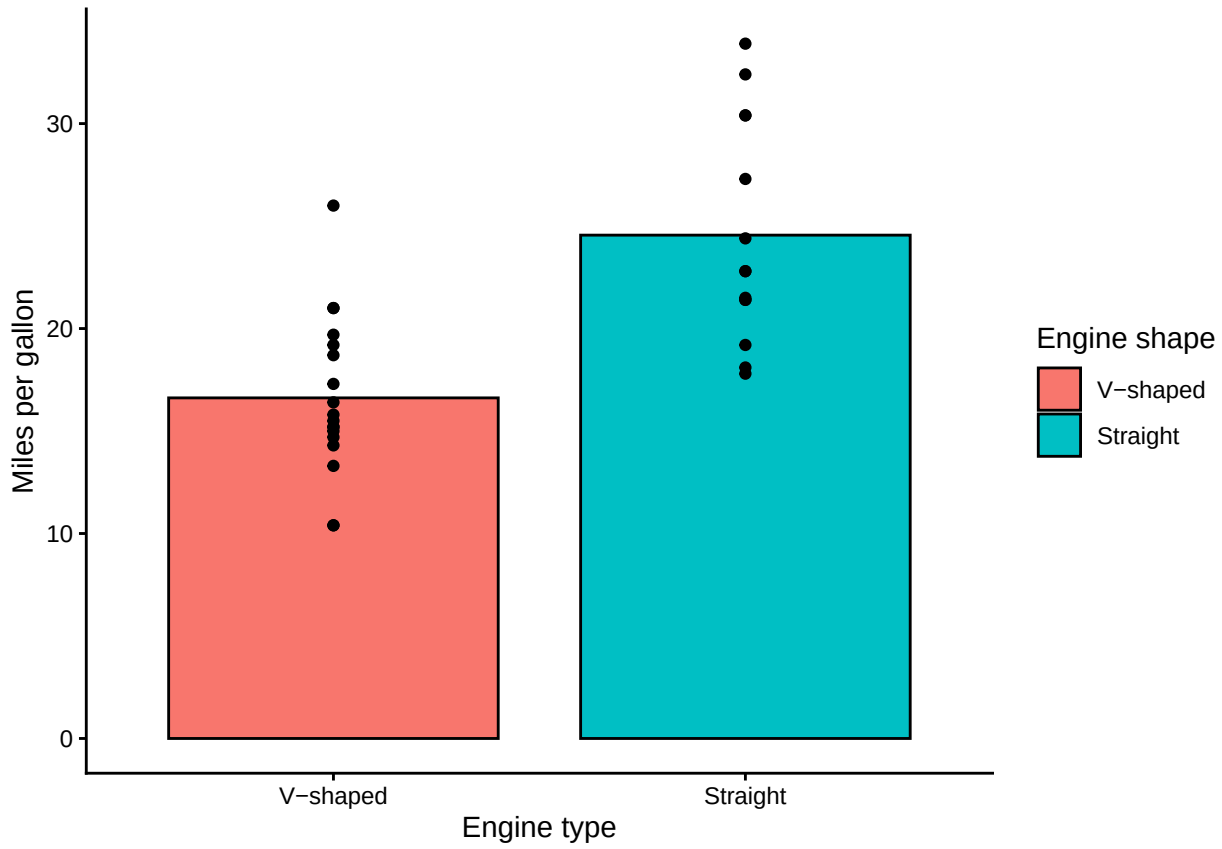
Based on these results is there a correlation between weight (1000 lbs) and mpg?

Plotting Resuts

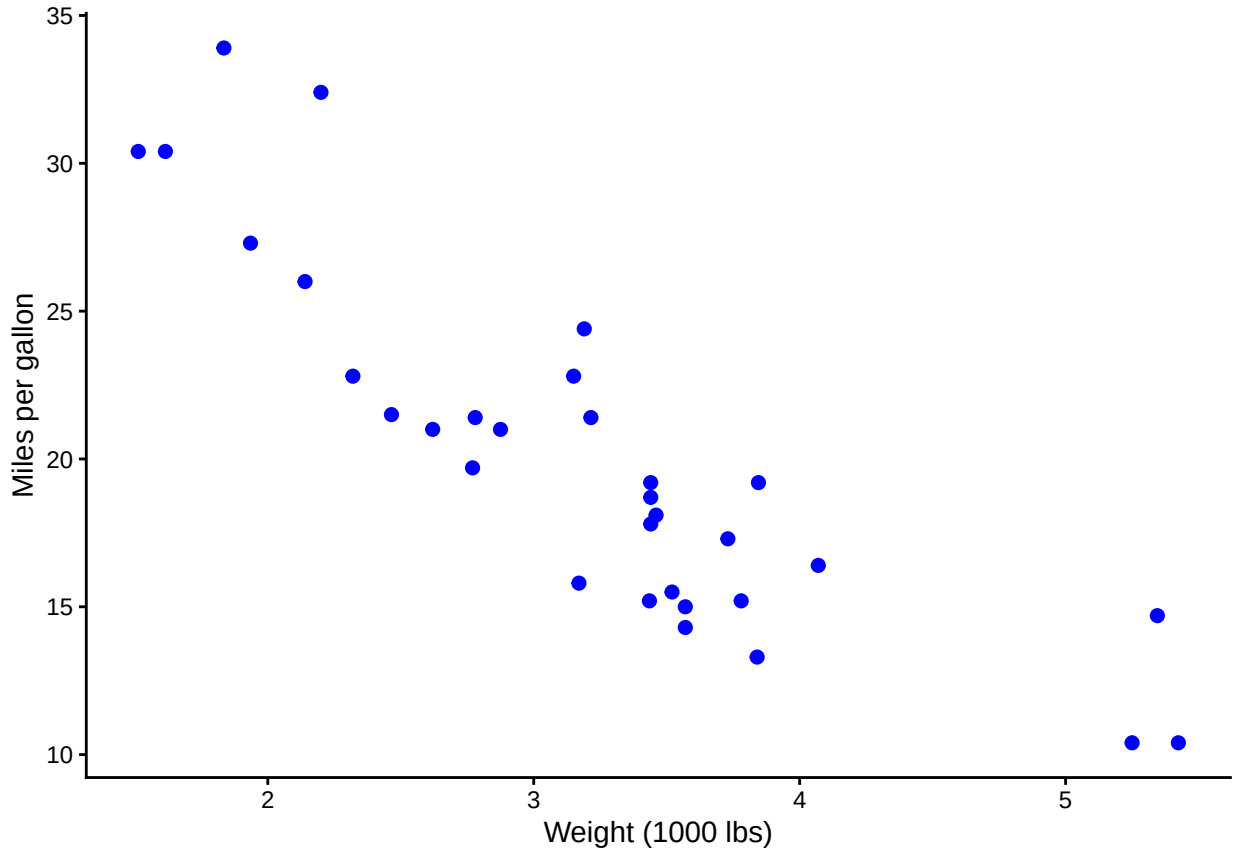
We can then plot these results using the ggplot2 package. This will be covered in a separate document but the graphs and code are available below.

```
# install first if you need to
library(ggplot2)

# Make barplot
ggplot(df, aes(x = as.factor(vs), y = mpg)) +
  geom_bar(aes(fill = as.factor(vs)), col = "black",
           stat = "summary",
           fun = mean,
           width = 0.8) +
  theme_classic() +
  labs(fill = "Engine shape",
       x = "Engine type",
       y = "Miles per gallon") +
  geom_jitter(position = position_dodge(width = 1)) +
  scale_x_discrete(labels = c("V-shaped", "Straight")) +
  scale_fill_discrete(breaks = c("0", "1"),
                      labels = c("V-shaped", "Straight"))
```



```r
# Make scatter plot
ggplot(df, aes(x = wt, y = mpg)) +
  geom_point(size = 2, col = "blue") +
  theme_classic() +
  labs(x = "Weight (1000 lbs)",
       y = "Miles per gallon")
```



Conclusion

Hopefully this document has provided you with a brief overview of using R programming for data analysis and you are now confident to have a go with your own data. The most important thing to remember when coding is that it is okay not to know how to do something, the real trick comes from knowing where to look for help.